RAPPRESENTAZIONE GRAFICA E COMMENTO

double ad[4]: $\overbrace{ad[0]}^{8\text{ byte}}$, $\overbrace{ad[1]}^{8\text{ byte}}$, ad[2], ad[3] ⟶ l'i-esimo elemento si trova all'indirizzo "base + i·8" (size of double)

float af[8]: $\overbrace{af[0]}^{4\text{ byte}}$, af[1], ..., af[7] ⟶ l'i-esimo elemento si trova all'indirizzo "base+i·4" → size of float

int ai[3]: $\overbrace{ai[0]}^{4\text{ byte}}$, ai[1], ai[2] ⟶ l'i-esimo elemento si trova all'indirizzo "base+i·4" → size of int

int x=1 (4 byte)

float y=1.1 (4 byte)

L'output del programma è: 0x.... 30
0x... f0
0

Il risultato è questo perchè si utilizza l'istruzione (&y)[1]=0. Si ha che x e y sono allocati in celle di memoria contigue e spostandosi in avanti di 4 byte oltrepassa i suoi limiti di memoria e si finisce dove inizia la x e ciò restituisce come risultato zero